

BS ARTIFICIAL INTELLIGENCE · PROGRAMMING FUNDAMENTALS

The Campus Analytics Engine

1. Project Overview

Students will build a multi-file, menu-driven data analytics system for a fictional university campus. The system manages student records, course enrollments, attendance, grades, and fee transactions — loading from TXT files, processing data using only fundamental C++ programming constructs, and producing formatted console reports.

⚠ AI Use Policy

Students may use AI assistants to understand concepts, debug errors, or generate small helper snippets — but they must be able to explain everything about the submitted code during viva.

2. Marks Breakdown (15 Marks Total)

5

MARKS

GitHub Repository

Commit history · multi-file structure
· README

5

MARKS

Implementation

All modules · correct output · PF
concepts used

5

MARKS

Viva Voce

Explain · trace · modify on the spot

i GitHub Grading Detail

Commit history will be inspected. A single large commit scores 1/5 regardless of code quality. Minimum requirement: one commit per completed module, plus a final integration commit. Commit messages must be descriptive — messages like 'update' or 'fix' will be penalized.

✗ Strictly Forbidden

#include <algorithm> — no sort(), find(), or any STL algorithm
#include <map> / <unordered_map> / <set> — use struct arrays and nested loops instead
class keyword — OOP is a separate course; use struct only
Monolithic submission — all logic in one .cpp file receives zero
Hardcoded data — system must read and write from txt files
#include <ctime> — date arithmetic must be done manually with string parsing

✓ Bonus Mark (+1 to Implementation, maximum 5)

Implement a 'search as you type' loop: user enters one character at a time in a while loop and the matching student list filters and reprints after each keystroke — demonstrating string prefix matching using only substr and length comparisons.

3. Required File Structure

The project must consist of exactly 8 source files plus 5 txt data files. Each .cpp file has a header file (.h) except main.cpp. Cross-file function calls are mandatory — submitting everything in one file results in zero marks for Implementation.

3.1 Source Files

#	File (.cpp / .h)	Responsibility	Depends On
M0	main.cpp	Entry point. 3-level nested menu system. Calls all modules.	All modules
M1	filehandler.h / .cpp	All txt read/write.	— (standalone)
M2	student_ops.h / .cpp	Add, search, update, and soft-delete students. Validate BSAI-YY-XXX roll format with string ops.	filehandler
M3	course_ops.h / .cpp	Enroll/drop courses, prerequisite check, credit load validation (max 21 hrs).	filehandler, student_ops
M4	attendance.h / .cpp	Mark attendance (P/A/L), compute %, flag <75% shortage, undo last session.	filehandler, course_ops
M5	grades.h / .cpp	Enter marks, compute weighted total, letter grade, semester GPA, and class statistics.	filehandler, student_ops, attendance
M6	fee_tracker.h / .cpp	Record payments, compute late fines (date arithmetic without ctime), and generate receipts.	filehandler, student_ops
M7	reports.h / .cpp	Merit list, attendance defaulters, fee defaulters, and formatted tabular output.	grades, attendance, fee_tracker

3.2 Data Files

File	Key Columns	Notes
students.txt	roll, name, dept, cgpa, status	status = active/inactive (soft delete)
courses.txt	code, title, credits, seats, prereq	prereq = course code or NONE
enrollments.txt	roll, course_code, semester, status	status = enrolled / dropped
attendance_log.txt	roll, course_code, date, status	status = P / A / L (late)
fees.txt	roll, semester, amount_due, amount_paid, due_date, paid_date	dates in DD-MM-YYYY format

4. Functional Specification

Below is the required function signature and specification for each module. Students must implement at minimum these functions; they may add helper functions as needed.

M1 — filehandler.h / filehandler.cpp

Function Signature	Specification
readTXT	Opens file, skips header row, parses each line into vector<string> using character loop (no getline split).
writeTXT	Opens file for overwrite, writes header row, then all rows. Each field is separated by a comma. Handles fields containing commas by wrapping in quotes.
appendTXT	Appends a single row to the existing file without loading it fully into memory.
findRow	Linear search through file rows, returns a matching vector<string> or an empty vector if not found.
rowExists	Returns bool — true if any row has a value at colIndex.

M2 — student_ops.h / student_ops.cpp

Function Signature	Specification
addStudent	Validates roll format BSAI-YY-XXX using substr and character checks (no regex). Checks for no duplicates. Validates name has no digits. CGPA in [0.0, 4.0]. Appends to students.txt.
searchByRoll	Returns student details or an empty if the student ID is not available.
searchByName	Get all students where name contains substring
updateStudent	Loads file, finds row, updates specified field (not roll), rewrites file.
softDelete	Sets status field to 'inactive'. Does not remove the row physically.
listActiveStudents	Returns all active students sorted by roll number using selection sort.

M3 — course_ops.h / course_ops.cpp

Function Signature	Specification
enrollStudent	Checks: student active, course exists, seats > enrolled count, student not already enrolled, credit load <= 21, prerequisite passed. Returns enum EnrollResult with success/failure reason.
dropCourse	Only permitted if no attendance rows exist for this student+course+semester. Updates enrollment status to 'dropped'.
getCreditLoad	Sums credit hours of all active enrollments for the given semester. Uses nested loop over enrollments and courses files.
checkPrerequisite	Looks up the prereq field of the course. If not NONE, checks the grades file to confirm the student has a non-F grade in that course.
listEnrolledStudents	Returns of all active enrolled students in a course.

M4 — attendance.h / attendance.cpp

Function Signature	Specification
markAttendance	Iterates enrolled students, prompts P/A/L for each. Saves a backup of the current attendance vector before writing. Appends rows to attendance_log.txt.
getAttendancePct	Computes $(\text{present} + 0.5 * \text{late}) / \text{totalSessions} * 100.0$ using an accumulator loop. Returns double.
getShortageList	Returns students with attendance < 75%
undoLastSession	Restores from backup vector (pre-session snapshot). Rewrites file. Returns false if no backup exists.
printDailySheet	Formatted console table of all enrolled students and their status for the given date.

M5 — grades.h / grades.cpp

Function Signature	Specification
enterMarks	Accepts up to 5 quiz marks (uses bestThreeOfFive), assignment array, mid (40), final (60). Validates ranges. Stores computed totals.
bestThreeOfFive	Finds and excludes the two lowest values using a loop (no sort). Returns the average of the remaining. Handles $n < 3$ edge case.
computeWeightedTotal	Applies: $\text{quiz} * 0.10 + \text{asgn} * 0.10 + \text{mid} * 0.30 + \text{final_} * 0.50$. Returns a double out of 100.
getLetterGrade	Maps: ≥ 85 A, ≥ 80 B+, ≥ 70 B, ≥ 65 C+, ≥ 60 C, ≥ 50 D, else F. Returns string.
computeGPA	Credit-weighted average of GPA points across all courses in a semester. Manual loop — no algorithm library.
computeClassState	Returns struct Stats {highest, lowest, mean, median}.
applyAttendancePenalty	If getAttendancePct < 75, overrides grade to F regardless of marks.

M6 — fee_tracker.h / fee_tracker.cpp

Function Signature	Specification
recordPayment	Validates date format DD-MM-YYYY with string checks. Update fees.txt paid amount.
computeLateFine	Gets due_date and paid_date from fees.txt. Calls daysBetween to get the difference. Calculates 2% per complete week.
daysBetween	Parses two DD-MM-YYYY strings, converts to total day count (manual month-length array, no ctime), returns integer difference.
generateReceipt	Prints formatted receipt: tuition breakdown, late fine if any, total due, paid, balance. Uses setw/setfill from iomanip.
getDefaulters	Returns students with a balance > 0 past due date. Sorted by outstanding amount using bubble sort.

M7 — reports.h / reports.cpp

Function Signature	Specification
<code>printMeritList</code>	All active students are sorted by CGPA in descending order. Formatted table with rank column.
<code>printAttendanceDefaulters</code>	Students with any course attendance < 75%. Shows roll, name, course, and percentage.
<code>printFeeDefaulters</code>	Call all fee defaulters. Formatted with outstanding amount and weeks overdue.
<code>printSemesterResult</code>	Full result sheet with borders. Grade, GPA, and attendance status per student.
<code>printDepartmentSummary</code>	Group students by dept field using nested loops and parallel arrays. Shows count, avg CGPA, pass rate per dept.
<code>exportReportToFile</code>	Redirects cout to ofstream for chosen report. Restores cout after. Saves .txt file.

5. Marking Rubric

5.1 GitHub (5 Marks)

Criteria	Full Marks	Partial	Minimal / Zero
Commit frequency & messages	8+ meaningful commits with descriptive messages	4–7 commits, some vague messages	< 4 commits or all vague / single dump commit
File structure in the repository	All 8 .cpp/.h + 5 data files present	Missing 1–2 files	Single-file submission or data files missing
README.md quality	Setup instructions, features list, sample run	Partial README	Empty or absent README

5.2 Implementation (5 Marks)

Criteria	Full Marks	Partial	Minimal / Zero
Modules complete and functional	All 7 modules compile and produce correct output	5–6 modules work; minor errors	< 5 modules or program crashes
PF concepts coverage	All required concepts are identifiable in the code	Minor gaps (1–2 concepts missing)	Core concepts absent (no file I/O, no sorting, etc.)
Error handling and input validation	No crashes on bad input; all validations work	Partial validation; some crashes	Crashes on common bad inputs
No forbidden constructs used	No STL algorithms, no class, no ctime, no map	Minor isolated use of forbidden lib	Core logic uses forbidden libraries

5.3 Viva Voce (5 Marks)

Criteria	Full Marks	Partial	Minimal / Zero
Explain any function on demand	Clear, confident, line-by-line explanation	Partial explanation with prompts	Cannot explain their own code
Trace execution with sample input	Correct step-by-step trace through logic	Rough trace with minor errors	Unable to trace execution
On-the-spot modification	Correct modification in < 5 minutes	Modifies with hints from instructor	Cannot modify code under guidance
Identify cross-module dependencies	Points to exact function call chain across files	Identifies roughly with errors	Cannot identify any cross-file calls

6. Submission Checklist

1. GitHub repository link submitted on LMS before the deadline
2. Repository contains all 8 .cpp/.h source files and 5 .txt data files
3. README.md explains how to compile and run the project
4. At least 8 commits with meaningful messages visible in the commit history
5. Code compiles without errors
6. No forbidden headers are present anywhere in any source file